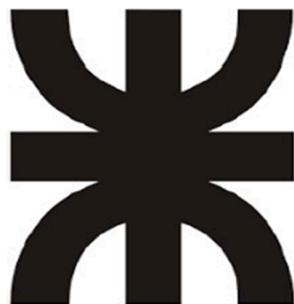


UNIVERSIDAD TECNOLÓGICA NACIONAL



FACULTAD REGIONAL ROSARIO

INFORME ACADEMIA

2025

ISI - 3K02

MATERIA: Tecnologías de Desarrollo de Software IDE

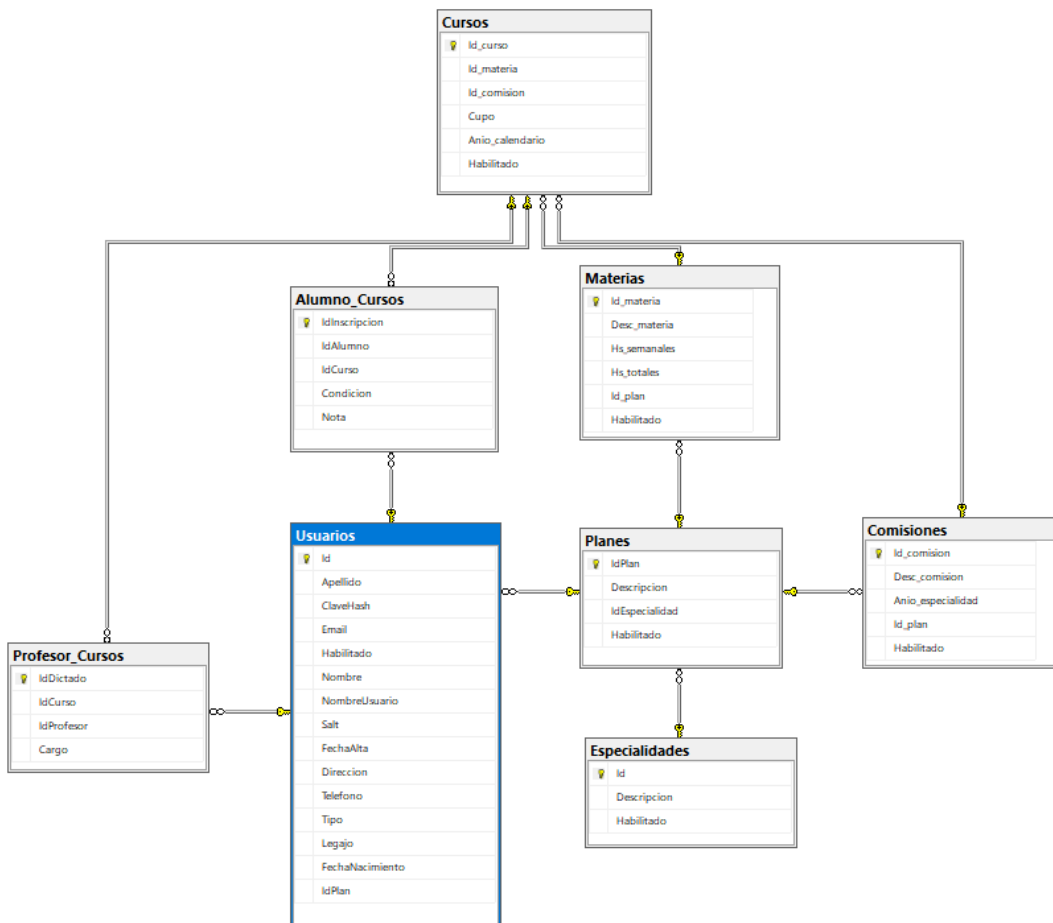
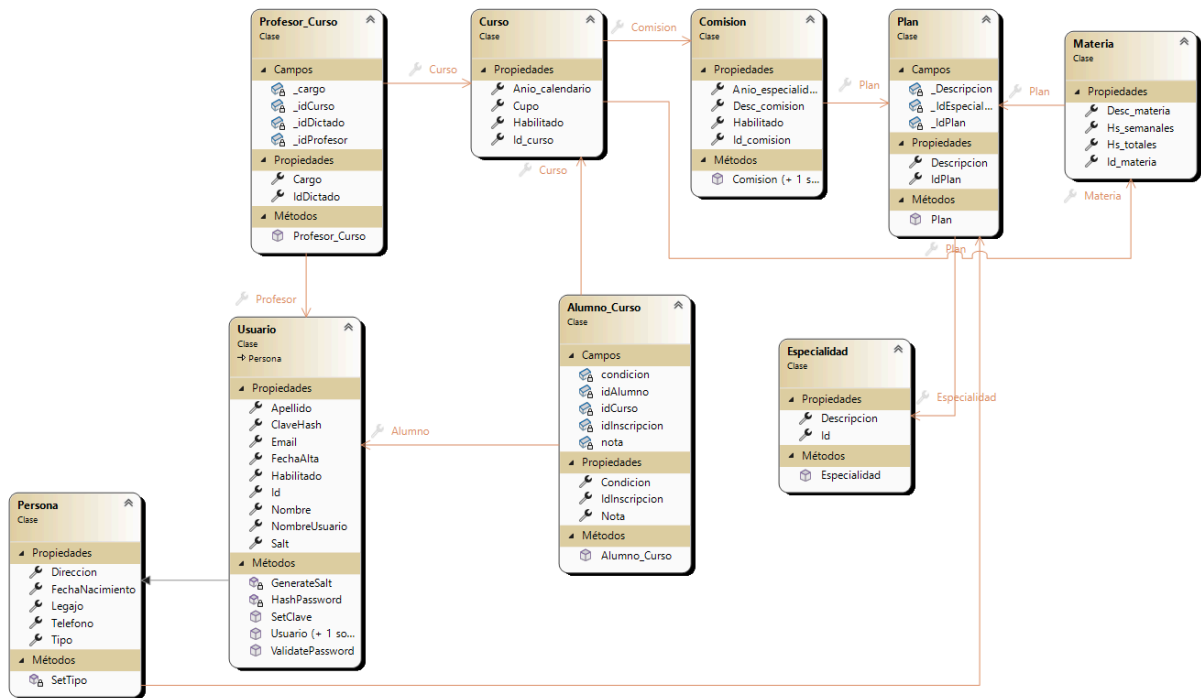
PROFESORES:

- Sebastián Rocco
- Ezequiel Porta

INTEGRANTES:

- Lurati, Ignacio
- Romero, Gabriel T.

Diagramas



Cosas a mejorar

- **Mejora en seguridad:** si bien para acceder a las funciones del WinForms y Blazor siempre hay que estar logueado y se valida el tipo de usuario que accede, en el back-end, por limitaciones técnicas y múltiples errores de validación de tokens, no están protegidos los endpoints. En un contexto académico es suficiente, sin embargo, en un desplegado real, cerrar los endpoints es indispensable.

- **Manejo de errores para el usuario:** informar al usuario sobre el error que ocurrió es importante, moderando el lenguaje técnico, y optando por ofrecer un mejor control de todas las alternativas. Entendemos que por limitaciones de alcance del proyecto, y al no operar con baja lógica en todas las entidades, ocurre que a veces, al tratar de eliminar una entidad cuya clave primaria es foránea de otra entidad, el error puede llegar a ser “poco descriptivo”. Optamos por no ofrecer este mensaje tan técnico, pero si podes cachearlo apropiadamente se dificulta mostrar todas las alternativas del mismo, además, lo más importante en mi opinión, no permitir dar de baja dichos elementos del sistema.

- **Inyección de dependencias:** si bien decidimos usar DI, la realidad es que está aprovechado “a medias”, pues las inyecciones suceden por el objeto en sí, y no por una interfaz del mismo. Esto quiere decir que al querer cambiar de versión de un objeto, llámese *repositorioA* a *repositorioA2.0*, sucede que hay que cambiar lo que recibe el endpoint al realizar la inyección, pues espera un objeto tipo *repositorioA*.

- **Exception Catching:** notamos que en los ejemplos, a la altura de los endpoints, muchas veces se utilizaba `ArgumentException` para manejar excepciones. Desconocemos el estándar de como suele resolverse esta clase de problemas, pero en Desarrollo de Software se nos recomendó usar un catch para CUALQUIER excepción producida, o lo que en .Net sería un catch de `Exception` directamente, evitando que se produzcan errores en EF que no puedan ser catcheadas y que detengan el flujo del programa.

- **Updates:** algunos updates no miden con firmeza que atributos se están cambiando. Si bien desde los ApiClient se envía la información correcta a cambiar, podría representar una vulnerabilidad de seguridad.

Descripción del sistema

Sistema de Academia, diseñado para que alumnos puedan inscribirse a su cursada, profesores puedan calificar y administrador puedan crear cursos, materias, planes, usuarios, etc.

El sistema consta de 3 usuarios posibles, alumno, profesor, administrador (estos últimos estando pre-cargados en el sistema). Un alumno tiene la posibilidad de ver notas de sus materias, el reporte de las mismas según el plan e inscribirse a alguna que considere. Un profesor tiene la posibilidad de ver sus cursos y de cargar notas a sus alumnos. Un administrador tiene la posibilidad de crear, modificar y eliminar todos los elementos del sistema, desde usuarios hasta materias, más la habilidad de imprimir reportes de Cursos.

- El administrador acceden desde WinForms.
- Los profesionales y alumnos acceden desde WinForms y Blazor.

Blazor

[About](#)

Iniciar Sesión

Nombre de Usuario *

Contraseña *

[Iniciar Sesión](#)

¿No tienes cuenta? [Registrate aqui](#)

Sistema Académico

About

Inicio

Mis Cursos

Perfil

Cerrar Sesión

Mis Cursos Asignados

Materia	Comisión	Año	Cargo	Acciones
Análisis Matemático	Comision A	2023	Titular	Ver Alumnos
Dibujo Técnico	Comision D	2024	Auxiliar	Ver Alumnos

Total de cursos: 2

Sistema Académico

About

Inicio

Mis Cursos

Perfil

Cerrar Sesión

Alumnos del Curso

Volver

Legajo	Nombre de Usuario	Condición	Nota	Acciones
10001	jmr	Regular	8	Modificar

Sistema Académico

About

Inicio

Mis Cursos

Perfil

Cerrar Sesión

Mi Perfil

Volver

Información Personal

Legajo

20007

Este campo no se puede modificar

Nombre *

Martin Alejandro

Nombre de Usuario *

martind

Dirección

San Martin 150

Fecha de Nacimiento

01/11/1978

Tipo de Usuario

profesor

Este campo no se puede modificar

Apellido *

Diaz

Email *

martin.d@email.com

Teléfono

3415566778

Los campos marcados con * son obligatorios

Cancelar

Guardar Cambios

Sistema Académico

About

Inicio

Mis Inscripciones

Inscribime

Perfil

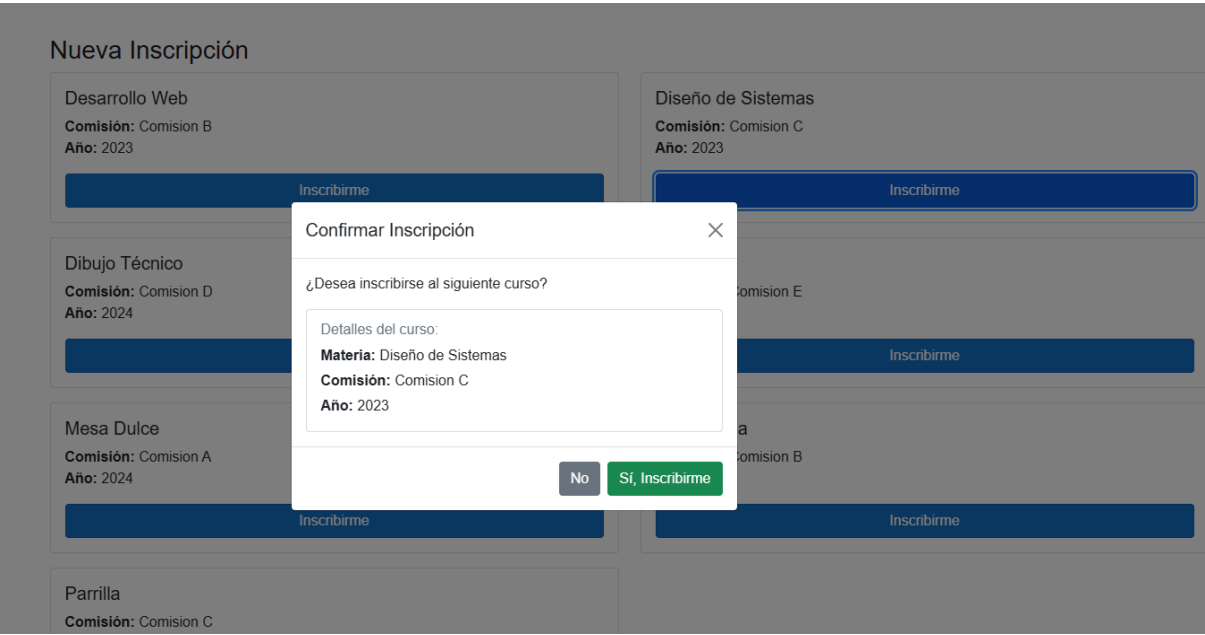
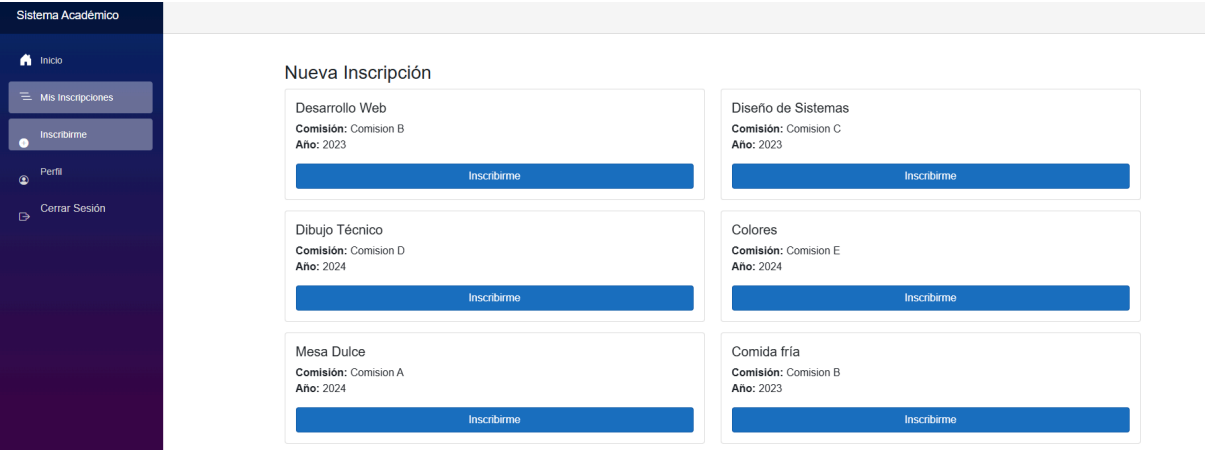
Cerrar Sesión

Mis Inscripciones

Generar Reporte Plan de Estudios

Materia	Comisión	Condición	Nota
Análisis Matemático	Comision A	Regular	8
Materiales	Comision F	Libre	-

Total de inscripciones: 2



ru14YWP1PjxJK6wf13XkMjwueSxnLAIIBOKZIOZmoQoorp...

1 / 1

100%

Universidad Tecnológica Nacional (UTN)

Materia: Tecnologías de Desarrollo de Software IDE (.NET)

Plan de Estudios - Juan Manuel Romero

Información del Alumno

Legajo: 10001

Nombre: Juan Manuel Romero

Información del Plan

Plan: Plan Familiar

Especialidad: Diseñador de interiores

Estadísticas

Total de Materias del Plan: 3

Materias Aprobadas: 0

Materias del Plan

Materia	Horas Semanales	Condición
Colores	3	
Dibujo Técnico	3	
Materiales	3	Libre

WinForms

Login

Iniciar Sesión

Usuario

Contraseña

Volver

Login

¿No tenés usuario?

[Registrate acá](#)

Cursos

	Id_curso	Anio_calendario	Cupo	Comision	Materia
▶	1	2023	30	Comision A	Análisis Matemático
	2	2023	25	Comision B	Desarrollo Web
	3	2023	20	Comision C	Diseño de Sistemas
	4	2024	35	Comision D	Dibujo Técnico
	5	2024	28	Comision E	Colores
	6	2023	18	Comision F	Materiales
	7	2024	22	Comision A	Mesa Dulce
	8	2023	30	Comision B	Comida fría
	9	2024	25	Comision C	Bevitas

Volver

Exportar PDF

Detalle

Eliminar

Agregar +

Detalle Curso

Materia:

Análisis Matemático

Comisión:

Comision A

Año calendario:

2023

Cupo:

30

Alumnos

Profesores

Agregar Profesor

	Legajo	Alumno	Condición	Nota
	10001	Juan Man...	Regular	8

	Legajo	Profesor	Cargo
	20007	Martín Alejand...	Titular

Cancelar

Modificar Curso

Guardar

Detalle Materia

Descripcion:

Diseño de Sistemas

Hs semanales:

2

Hs totales:

32

Plan:

Plan Premium

Cancelar

Guardar



Universidad Tecnológica Nacional (UTN)
Materia: Tecnologías de Desarrollo de Software IDE (.NET)

Reporte de Curso - Análisis Matemático

Comisión: Comisión A | Año: 2023

Información del Curso

Cupo Total: 30 Inscriptos: 1 Disponibles: 29

Estadísticas de Notas

Promedio General: 8,00 Aprobados: 0 Sin Nota: 0
Total con Nota: 0 Reprobados: 0

Distribución de Notas

Porcentaje por Condición

Distribución por Rango de Notas

0-3	4-6	7-10
0 alumnos	0 alumnos	1 alumnos

Profesores Asignados

Legajo	Nombre	Cargo
20007	Martin Alejandro Diaz	Titular

Listado de Alumnos

Legajo	Alumno	Condición	Nota
10001	Juan Manuel Romero	Regular	8

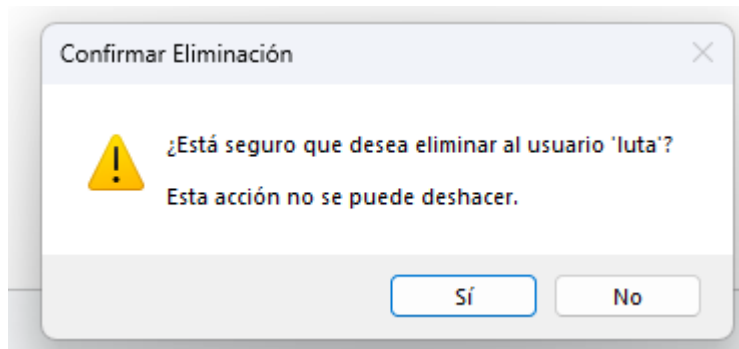
RegisterForm

Registrar Usuario

Nombre		Plan	
Apellido		Tipo	profesor
Nombre de Usuario		Dirección	
Email		Teléfono	
Contraseña		Fecha de Nacimiento	31/10/2025

Volver

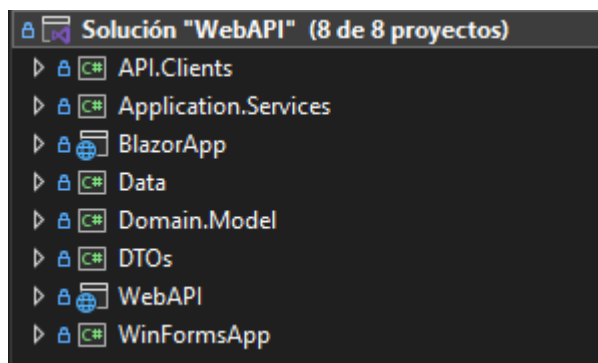
Registrar



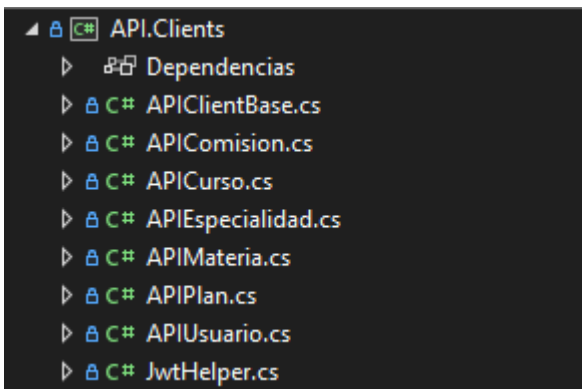
Arquitectura

Arquitectura empleada: MinimalAPI

Estructura principal:

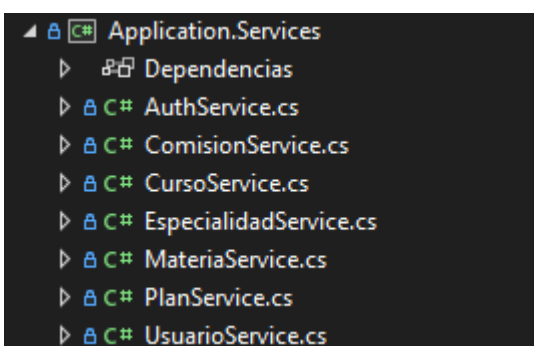


Api Clients:



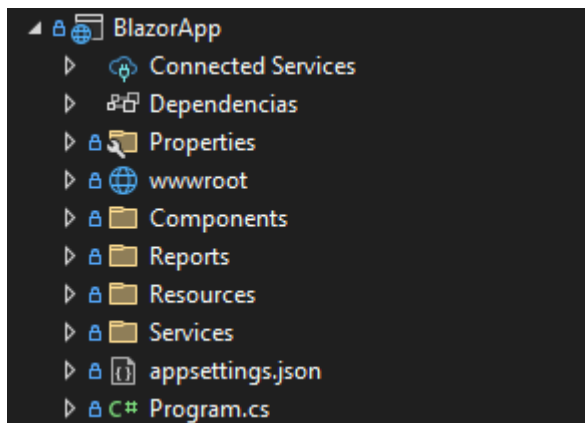
Encargada de realizar las comunicaciones con la WebApi, genera clientes Http con métodos asíncronos. Todas heredan de ApiClientBase, quien recibe y reenvia el token en cada petición.

Applications Services:



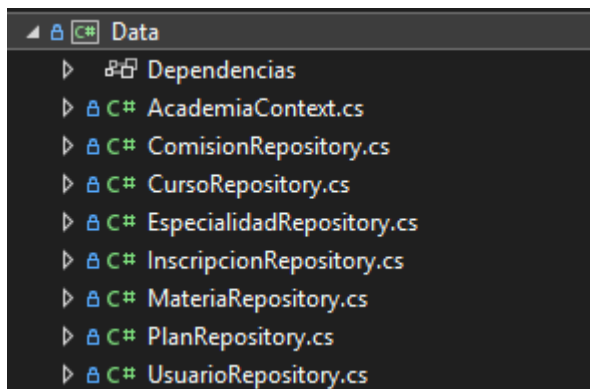
Capa destinada a resolver lógica de negocio. Es invocada en el program, tiene acceso directo a los repositorios inyectados, DTOs y objetos del modelo. Realiza validaciones y mapea objetos a DTOs.

BlazorApp:



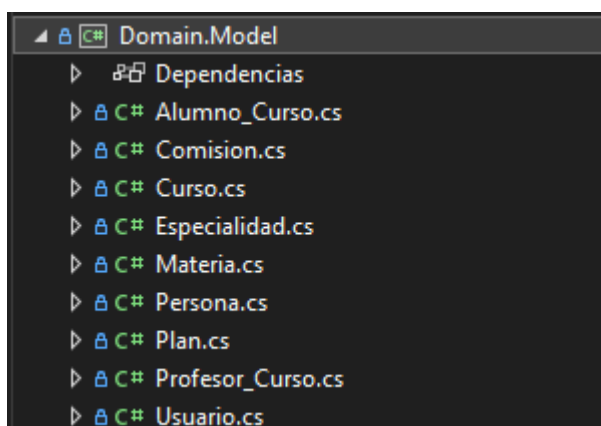
Capa de presentación de Blazor. Incluye componentes, ya sean reutilizables o páginas, resources (como imagenes), servicios para realizar reportes y sus launchSettings. Tiene acceso directo a DTOs y a los APIClients.

Data:



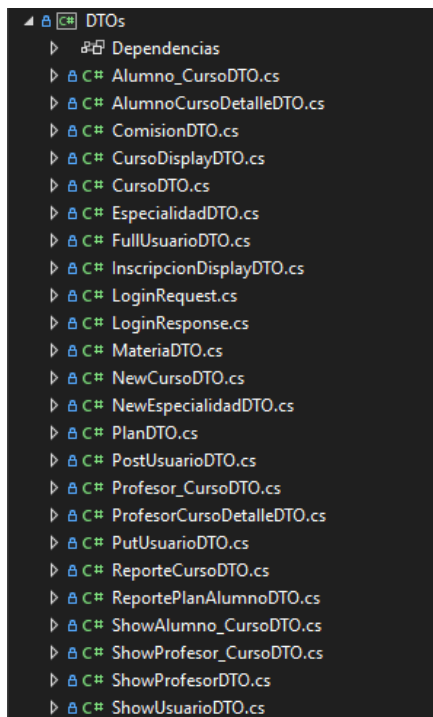
Maneja los repositorios, los cuales acceden al context de la academia, el cual posee desde la configuración de EntityFramework hasta la creación de las entidades en la BD. Posee acceso directo a la capa de modelo y es solo invocable por un servicio.

Domain Model:



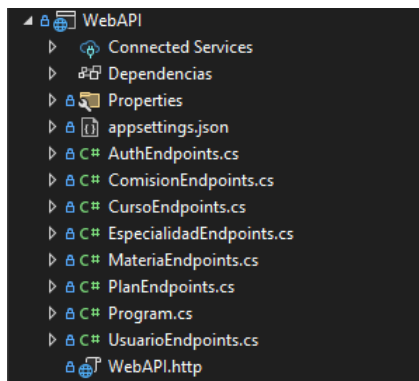
Define los atributos y propiedades de los objetos del sistema, junto a sus niveles de acceso, herencia, relaciones, validaciones críticas y formas de ser creados, tanto para EF como para los servicios.

DTOs:



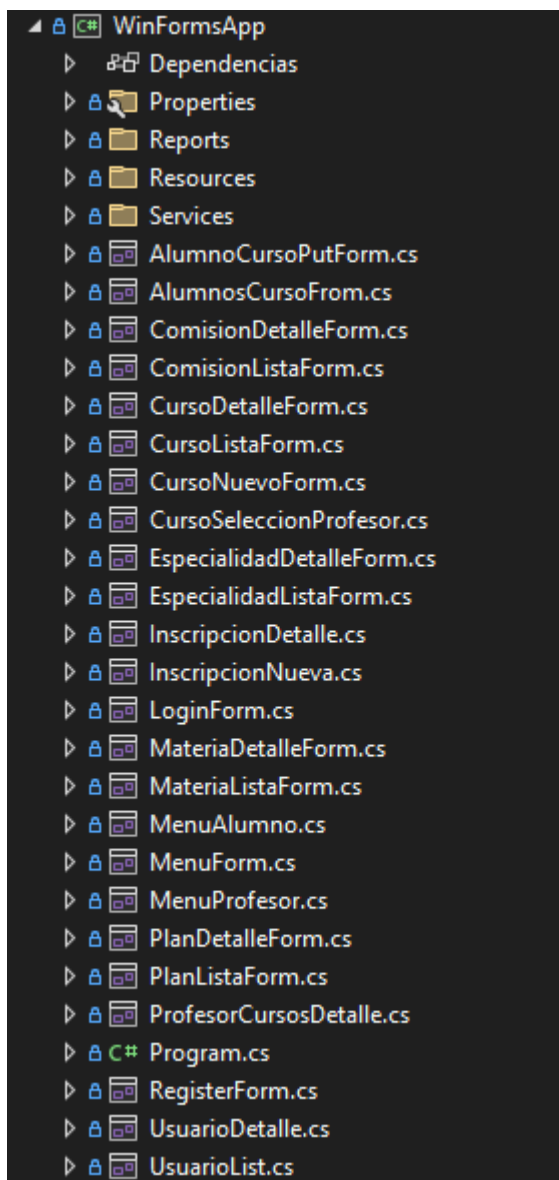
Son los objetos planos que permite comunicar todas las capas del sistema hasta finalmente formar el objeto final en los repositorios. Se utilizan en todas las capas de presentación, en los endpoints y en los servicios. Solo definen la información necesaria para el transporte de datos, no implementan validaciones ni lógica de negocio.

WebApi:



Provee del Swagger, middlewares de autenticación, los endpoints del sistema, la inyección de servicios, ya sea el AcademiaContext, los repositorios o los Services. Permite la comunicación con otros sistemas vía API Rest junto a DTOs. Se encarga de invocar los servicios requeridos para resolver el caso de uso.

WinForms:



Posee los formularios de la vista. Tiene acceso directo a DTOs y a APIClients, usando este último para llamar a WebApi. El actual proyecto de arranque es LoginForm.cs.

Usuario => WinForms / Blazor => ApiClients => WebApi => Service => Data

Para comunicar sus datos, WinForms/Blazor, ApiClients, WebApi y Service emplean *DTOs*, mientras que Service (lo usa para enviarle a Data) y Data emplean el *Domain Model*.

Tecnologías utilizadas

- **System.IdentityModel.Tokens.Jwt** 8.1.0
- **Microsoft.AspNet.WebApi.Client** 6.0.0
- **Microsoft.Extensions.Configuration.Abstractions** 9.0.9
- **Swashbuckle.AspNetCore** 9.0.3
- **MudBlazor** 8.13.0
- **QuestPDF** 2025.7.3
- **Microsoft.EntityFrameworkCore** 9.0.9
- **Microsoft.EntityFrameworkCore.SqlServer** 9.0.9
- **Microsoft.EntityFrameworkCore.Tools** 9.0.9
- **Microsoft.AspNetCore.Authentication.JwtBearer** 8.0.0
- **Microsoft.AspNetCore.OpenApi** 7.0.18

Usuarios de prueba

Administradores:

gtr - 123456

luta - 123456

Alumno:

jmr - 123456

Profesor:

martind - 123456

NOTA: Existen muchos usuarios más, pero los mencionados tienen datos pre-cargados que agilizan las pruebas. Para más información consultar el `entity.HasData()` de `Usuario` en el `AcademiaContext`.